Recruitment Pipeline Performance & Throughput Improvements

Generator -> PostgreSQL/WAL -> Debezium -> Kafka -> Spark Match Engine -> Delta

Consolidated report of the changes made to improve runtime, throughput, stability, and batch efficiency

Executive Summary

Area	Main improvement	Performance meaning	Evidence status
Generator	PostgreSQL-only writer + one persistent DB connection + in-memory lifecycle pools	Less network/serialization work in the generator; avoids reconnecting; avoids repeated whole-table state discovery	Implemented in saved generator
Match engine	Native Spark SQL scoring instead of Python UDFs	Removes JVM <-> Python worker round-trips and lets Catalyst/codegen optimize scoring	Implemented in optimized match engine
Batch processing	Cache reused DataFrames, especially incoming/resolved/scored	Prevents repeated execution of the same join/filter/score lineage	Explicitly identified in code comments as major optimization
Joins	Column pruning + broadcast candidate/job dimensions	Reduces data volume and avoids expensive shuffle-style joins for small dimensions	Implemented
Actions / metrics	Avoid unnecessary counts and use Delta transaction metrics	Reduces extra Spark jobs while still exposing throughput/row/file metrics	Implemented
Reliability	Worker reuse, local[2], pre-created Delta tables, Windows-safe parquet setting	Fewer process-spawn/native-library failures, which prevents retries/crashes from destroying throughput	Implemented

Bottom line: the biggest performance gains are in the Spark match/batch path, not the scoring rules themselves. The saved optimized engine explicitly targets duplicate lineage execution, Python UDF overhead, unnecessary scans/actions, and dimension-join cost. The generator is improved architecturally, but the current generator still performs one SQL transaction/commit per operation, so it remains a major end-to-end throughput bottleneck at million-operation scale.

Evidence: recruitment_lifecycle_generator.py, lines 29-41 and 89-97; match_engine(1).py, lines 100-125, 290-345, 1947-2065, 2306-2898.

1. Scope and Methodology

Artifacts reviewed: the saved recruitment lifecycle generator, the current optimized match engine, the older match engine, and the stored Spark execution log from 25 Aug 2026. A standalone file named batch_processing.py was not found in the saved project library. The batch-processing logic is implemented inside match_engine.py as process_application_batch(), so this report treats that function as the project's batch-processing component.

- The report distinguishes direct performance optimizations from stability/correctness changes that indirectly protect throughput.

- No exact percentage speedup is claimed because the saved project does not contain a controlled before/after benchmark with identical workload and hardware.
- Where the source code itself labels a change as a major optimization, that is noted explicitly.

Evidence: File inventory: recruitment_lifecycle_generator.py; match_engine.py; match_engine(1).py; saved Spark log " نص واحد ملصق .txt".

2. End-to-End Performance Picture

Stage	Primary work	Main performance risk	Optimization focus
1. Generator	Create candidates, jobs, applications; write to PostgreSQL	Per-row SQL + commit overhead	Persistent connection; in-memory lifecycle state; workload-aware operation weights
2. PostgreSQL / WAL	Persist changes and expose CDC	Database transaction overhead	Keep CDC out of application code; let Debezium read WAL
3. Debezium -> Kafka	Capture row-level changes and publish events	Serialization / network throughput	Explicit downstream CDC boundary; Spark consumes streams
4. Match engine ingestion	Parse Debezium envelopes; convert timestamps	Schema inference / repeated parsing	Explicit StructType schemas + direct from_json projection
5. Dimension writers	Upsert candidate_dim and job_dim	Duplicate CDC rows / Delta MERGE ambiguity	Deduplicate by key/version and cache merge source
6. Application batch	Combine new + pending apps and enrich	Repeated scans and joins	Cache + broadcast + column pruning
7. Scoring	Calculate 5 component scores and overall priority	Python UDF overhead	Native Spark SQL expressions
8. Output	Write Delta + publish HIGH_PRIORITY to Kafka	Extra counts / re-computation	Delta operationMetrics + cached scored DF

3. recruitment_lifecycle_generator.py - Changes That Help Throughput

3.1 PostgreSQL-only architecture

Change: the generator was deliberately reduced to a pure PostgreSQL writer. It no longer handles Kafka directly; Debezium reads PostgreSQL WAL and publishes candidate_events, job_events, and application_events.

Why it helps: the generator avoids doing database writes plus Kafka production plus application-level CDC envelope work in the same hot loop. This makes the generator simpler and lets CDC operate asynchronously downstream.

Evidence: recruitment_lifecycle_generator.py, lines 29-41 and 31-37 in the saved artifact.

3.2 One persistent PostgreSQL connection

The main function opens PostgreSQL once, creates LifecycleEngine(conn), executes the full operation loop, and closes the connection only after all configured operations finish. This avoids connection establishment/teardown for every generated event.

Evidence: recruitment_lifecycle_generator.py, lines 135-145 and 223-230; connect_postgres at lines 89-97.

3.3 In-memory lifecycle state

LifecycleEngine hydrates active/inactive candidates, active/closed jobs, and active/final applications once at startup, storing them in dictionaries/sets. The operation chooser then checks those in-memory pools before selecting an operation. This removes repeated full-table discovery for entity eligibility.

Evidence: recruitment_lifecycle_generator.py, lines 532-576; choose_operation at lines 102-129.

3.4 Weighted workload generation

The generator uses a workload-oriented operation mix. In the saved configuration, total_operations is 1,000,000 and application inserts have weight 150 while application updates have weight 15. The code comments estimate roughly 77.9% application inserts and 11.8% application updates. This is primarily a workload-shaping optimization: it keeps the test representative of the intended high-volume application stream instead of spending equal work on low-volume entity types.

Evidence: recruitment_lifecycle_generator.py, lines 69-86.

3.5 Preloaded reference data

Generation functions operate from imported in-memory lists/dictionaries such as job_titles, jobs_skills, location, salary ranges, and experience mappings. There are no per-row reads from external reference files inside the generation functions.

Evidence: recruitment_lifecycle_generator.py, lines 56-67 and generator functions at lines 339-428.

3.6 Generator limitations that remain

- Each INSERT still executes one SQL statement followed by conn.commit().
- Lifecycle updates still perform row reads, writes, and commits one operation at a time.
- The current application selection still chooses candidate and job independently using random.choice(), so this was not a performance fix and does not by itself create realistic candidate-job affinity.
- There is no executemany()/COPY-based batch insert or commit grouping in the saved generator. This is the single clearest remaining generator-side optimization opportunity.

Evidence: recruitment_lifecycle_generator.py, insert_job_post/insert_candidate/_insert_application_db at lines 434-511; insert_application at lines 945-990.

4. match_engine.py / match_engine(1).py - Major Spark Optimizations

4.1 Replace Python UDF scoring with native Spark SQL expressions - highest-impact change

Before: the older engine used Python UDFs for skill, experience, salary, location, language, and priority scoring.

After: the optimized engine expresses those rules with built-in Spark SQL functions such as transform, lower, trim, array_intersect, size, regexp_extract_all, array_min/array_max, split, element_at, when, otherwise, and create_map.

Why it is faster: the saved code comments explicitly identify the JVM -> Python worker -> JVM round-trip and per-row serialization as a major cost. Native expressions remain in the Spark JVM/Catalyst execution path and can benefit from code generation.

Evidence: match_engine(1).py, historical optimization notes at lines 1402-1416; active native scoring functions at lines 1947-2065.

4.2 Cache the resolved applications DataFrame

The optimized batch caches resolved immediately after the candidate/job joins and filters. This was explicitly described in the code as the "biggest one from the log analysis": previously, resolved was materialized for an emptiness check and then recomputed again when scoring. The cache makes the check and scoring reuse the same computed lineage.

Evidence: match_engine(1).py, lines 780-786 and historical optimization note at lines 582-595 in the saved artifact.

4.3 Cache the scored DataFrame

After scoring, scored is cached before the results write and before the HIGH_PRIORITY filter. The source comment explicitly states that the old version could recompute scoring when checking high-priority rows, with an observed roughly 3-second HIGH_PRIORITY emptiness cost even when there were no results. Caching avoids that duplicate scoring lineage.

Evidence: match_engine(1).py, lines 1018-1020 and lines 1057-1104.

4.4 Cache incoming new + pending applications

incoming combines new CDC applications with pending retries and is immediately cached. It is then reused by the emptiness check and the dimension enrichment joins. This removes repeated execution of the union/filter lineage.

Evidence: match_engine(1).py, lines 641-662.

4.5 Broadcast candidate and job dimensions

The engine reads the dimensions and uses F.broadcast(candidate_dim) and F.broadcast(job_dim) for the enrichment joins. This is appropriate when the dimension snapshots are small enough for broadcast and avoids the more expensive shuffle-based join path for the large application-side stream.

Evidence: match_engine(1).py, lines 755-768.

4.6 Column pruning before broadcast/cache

The dimension reads no longer pull the entire candidate/job rows. Candidate data is restricted to candidate_id, name, skills, experience_range, current_salary, location, nationality. Job data is restricted to post_id, company, job_title, skills_needed, years_experience, salary, city, country, language, work_mode. The effect is smaller broadcast payloads, smaller cached objects, and less I/O.

Evidence: match_engine(1).py, candidate columns at lines 668-688; job columns at lines 696-721.

4.7 Cache dimension DataFrames

Candidate and job dimensions are aliased and cached so the broadcast join and their row-count metrics can reuse the same materialization rather than trigger additional Delta scans.

Evidence: match_engine(1).py, lines 731-749.

4.8 Avoid unnecessary Spark actions / counts

The optimized engine uses DataFrame.isEmpty() for lightweight emptiness checks, and it intentionally avoids count() on unresolved data. For row/file metrics after Delta writes, it reads Delta operationMetrics from transaction history instead of launching another Spark scan. Counts that remain are performed on already-cached DataFrames, so they are cache hits.

Evidence: match_engine(1).py, lines 286-287; process_application_batch lines 855-932; metrics design notes lines 1593-1606 in the saved artifact.

4.9 Deduplicate CDC rows before Delta MERGE

Dimension batches can contain multiple CDC records for the same key, especially during the first earliest-offset replay. The optimized upsert keeps only the highest-version row per key using row_number over

Window.partitionBy(key).orderBy(version desc), and it deduplicates deletes. This reduces unnecessary MERGE input and prevents Delta multiple-source-row ambiguity.

Evidence: match_engine(1).py, lines 290-345.

4.10 Pre-create Delta tables without starting Spark tasks

The engine pre-creates candidate_dim, job_dim, pending_applications, and dead_letter_applications with DeltaTable.createIfNotExists(...). The implementation deliberately avoids creating an empty Spark DataFrame from a Python list/RDD for schema-only initialization because even an empty PythonRDD can start Python worker processes on Windows.

Evidence: match_engine(1).py, lines 265-275 and the PENDING_APPLICATIONS_SCHEMA notes at lines 194-224; pending/dead-letter initialization in start_matching_query.

4.11 Python worker reuse and limited local concurrency

The optimized Spark session sets spark.python.worker.reuse=true and defaults to local[2]. The code comments explain that local[*] on Windows can spawn many Python processes simultaneously; lowering concurrency and reusing workers reduces process-launch overhead and failure probability.

Evidence: match_engine(1).py, lines 53-75 and lines 100-125.

4.12 Windows-safe Parquet setting

The active optimized build sets parquet compression to uncompressed to bypass the native Snappy DLL extraction path that was implicated in repeated Windows worker crashes. This is primarily a reliability optimization, not a guaranteed raw I/O speed optimization; the trade-off is larger files.

Evidence: match_engine(1).py, lines 66-75 and 100-125.

4.13 Delta write optimization settings

The Spark session enables Delta optimizeWrite and autoCompact through the properties.defaults.* configuration form. The source warns that effectiveness should be validated through files_written metrics. This targets the small-file problem and can reduce downstream file-management overhead.

Evidence: match_engine(1).py, lines 116-124.

4.14 Explicit schemas for Debezium events

The engine uses explicit StructType schemas for candidate, job, and application payloads plus an explicit Debezium envelope. This avoids large-scale schema inference and makes the streaming parser deterministic. It is mainly a correctness/overhead-control improvement and reduces the risk that nulls or arrays cause schema inference problems.

Evidence: match_engine(1).py, lines 127-263; older match_engine.py lines 219-262.

5. Batch Processing - process_application_batch()

Step	Implemented change	Why it helps
1. Input filter	Keep only c/r application events + required columns	Reduces records and payload carried into the batch
2. Early empty check	Return immediately when no new apps / no incoming apps	Avoids dimension reads and joins for empty work
3. Incoming cache	Cache new + pending union	Prevents duplicate union/filter evaluation
4. Dimension pruning	Read only join/scoring/publish columns	Shrinks Delta read and broadcast size

5. Dimension cache	Cache candidate/job snapshots	Avoids duplicate Delta scans when reused
6. Broadcast joins	Broadcast candidate + job dimensions	Avoids expensive shuffle join when broadcast is suitable
7. Resolved cache	Cache join result after presence filter	Prevents re-running enrichment lineage
8. No extra unresolved count	Use write + Delta transaction metrics	Avoids a separate scan just to count pending rows
9. Scored cache	Cache scoring output	Prevents re-running score logic for results + HIGH_PRIORITY
10. High-priority cache only when needed	Cache the small result set only after it is known non-empty	Avoids storing empty/small intermediate unnecessarily
11. Controlled cleanup	Unpersist all cached frames in _finish()	Prevents memory/cache accumulation across batches

Evidence: match_engine(1).py, process_application_batch active implementation around lines 2306-2898; especially incoming cache 2400-2460, dimensions 2485-2550, resolved cache 2570-2586, scoring cache 2680-2700, result/high-priority flow 2708-2805.

5.1 Early-exit strategy

The batch exits early when there are no new applications and again when no incoming applications remain. That means an empty/no-op cycle does not proceed into candidate/job dimension reads, joins, scoring, or writes.

Evidence: match_engine(1).py, process_application_batch lines 589-607 and 641-657.

5.2 Pending and dead-letter handling

Unresolved applications are retained for retry up to MAX_ENRICHMENT_RETRIES, while repeatedly unresolved rows are moved to a dead-letter Delta table. This is primarily correctness and operational resilience, but it also prevents endlessly reprocessing permanently broken records inside every batch.

Evidence: match_engine(1).py, lines 796-842 and 883-902.

6. What the Saved Runtime Logs Tell Us

The 25 Aug 2026 log is especially useful because it shows the actual shape of the workload rather than only code comments.

Observed log evidence	Interpretation
Batch 0 had ~14,087 job dimension upserts and ~6,298 candidate dimension upserts	Initial/replay dimension ingestion is a heavy part of first-batch cost
The engine reported "Current batch is falling behind" after ~30.049 seconds	The actual batch work exceeded the 30-second trigger interval during the initial workload
The log shows incoming/resolved caching steps before joins/scoring	The optimization was actually present in the running code path
Candidate/job dimensions were cached before broadcast joins	The dimension snapshots were designed to be reused rather than re-read

Evidence: Saved Spark log "نص واحد ملصق .txt", lines 284-300.

Important interpretation: the warning does not mean the optimizations failed. The first batch is a large replay/initialization workload, and the log shows significant dimension-upsert work before the matching path. The optimized code reduces redundant downstream work, but it cannot make a large initial Delta/Kafka replay free.

7. Before vs After Execution Path

Before (older engine)	After (optimized engine)
Python UDFs for scoring	Native Spark SQL scoring expressions
More repeated lineage evaluation	Cache incoming/resolved/scored when reused

Broad dimension reads	Select only required dimension columns
Implicit/regular joins	Explicit broadcast joins for small dimensions
RDD-based emptiness patterns in older path	DataFrame.isEmpty()
Extra row counts could trigger new scans	Use Delta operationMetrics and cached counts
Repeated source rows could reach MERGE	Deduplicate by key/version before MERGE
Schema-only table creation could hit PythonRDD	Create Delta tables directly through DeltaTable API
Python workers could be repeatedly spawned	Worker reuse + lower local concurrency
Snappy native DLL crash path on Windows	Uncompressed Parquet in active optimized build
Less explicit runtime instrumentation	Per-batch metrics: duration, throughput, rows, files, shuffle

Evidence: Older match_engine.py and optimized match_engine(1).py comparisons; see saved files listed in the scope section.

8. Performance Impact Ranking

Rank	Change	Expected impact	Confidence
1	Native Spark SQL scoring instead of Python UDFs	Very high - removes per-row JVM/Python worker boundary	High
2	Cache resolved applications	Very high - explicitly identified as biggest log-analysis optimization	High
3	Cache scored applications	High - avoids re-running score logic for high-priority detection	High
4	Broadcast + column-pruned dimensions	High - smaller payloads and avoids expensive join exchange when dimensions fit broadcast	High
5	Avoid unnecessary Spark actions / use Delta transaction metrics	High - fewer scans/jobs per batch	High
6	Cache incoming + dimension DataFrames	Medium-high - prevents duplicate upstream work	High
7	CDC deduplication before MERGE	Medium - reduces merge work and replay ambiguity	High
8	Generator persistent connection + in-memory lifecycle state	Medium - removes avoidable application-side overhead	High
9	Python worker reuse / local[2] / pre-created Delta tables	Medium indirect - mainly protects throughput from startup/crash overhead	High
10	Delta optimizeWrite/autoCompact	Potential medium - depends on file pattern and workload	Medium

Impact is relative and qualitative. It is not a measured percentage speedup.

9. Metrics We Added / Should Use to Prove Faster Runtime

Metric	What it tells us	How to use it
batch_duration_sec	End-to-end processing time per micro-batch	Primary latency KPI
input_rows_per_sec	Throughput	Primary throughput KPI
incoming_rows	Actual workload size	Normalize batch durations
resolved_rows / unresolved_rows	Enrichment success	Detect data-dimension lag
scored_rows	Rows that reached scoring	Compare against incoming
high_priority_rows	Business output volume	Detect scoring distribution issues
candidate_dim_broadcast_mb / job_dim_broadcast_mb	Broadcast payload size	Watch for broadcast becoming too large
files_read / files_written	Storage/file pressure	Detect small-file or scan problems
shuffle_read_mb / shuffle_write_mb	Data movement	High values indicate join/aggregation pressure

pending_write_sec / scoring_write_sec / resolved_check_sec / high_priority_check_sec	Stage-level wall time	Find the current bottleneck instead of guessing

Evidence: match_engine(1).py, BatchMetrics fields and timers in the optimized implementation; metrics design notes around lines 2142+ in the saved file.

9.1 Recommended benchmark protocol

- Run the same database state, Kafka offsets, Delta paths, Spark settings, and operation count for each comparison.
- Measure at least 3 warm runs after one initial run, because the first replay has different costs.
- Compare median batch_duration_sec and input_rows_per_sec, not a single best run.
- Separate initial replay performance from steady-state streaming performance.
- Track files_read, files_written, and shuffle MB so a faster run is not simply trading runtime for unhealthy storage behavior.

10. Remaining Bottlenecks / Next Highest-Value Improvements

Priority	Remaining issue	Recommended improvement	Expected effect
P0	Generator commits each operation individually	Batch INSERT/UPDATE transactions (e.g. 500-5,000 ops per commit) or use PostgreSQL COPY where lifecycle semantics allow	Likely largest generator-side throughput gain
P0	Generator performs SELECT -> UPDATE -> SELECT for many lifecycle operations	Use UPDATE ... RETURNING and keep the returned row/version in one round trip	Reduce database round trips
P1	Application candidate/job selection remains independent random choice	Use category/skill-aware candidate-job affinity while keeping a bounded candidate/job pool	Improves test validity; may also make downstream scoring distribution more useful
P1	Full dimension snapshots are re-read each application batch	Refresh/cache dimensions on a controlled interval or use a stateful dimension cache where correctness allows	Lower repeated Delta read cost
P1	Broadcast may eventually become too large	Monitor broadcast MB; add size guard or alternative join plan when dimensions exceed safe threshold	Prevents broadcast memory pressure
P2	Initial earliest-offset replay is heavy	For benchmark runs, use a controlled baseline checkpoint or pre-warmed dimensions	Separates first-load cost from steady-state KPI
P2	Parquet is uncompressed on Windows	Re-enable compression after fixing the native-library/AV issue, then benchmark file size vs write time	Potential storage and I/O improvement
P2	30-second trigger is a stability choice, not a direct speedup	Tune trigger only after batch duration is consistently below the target SLA	Lower latency once processing headroom exists

10.1 Trigger interval clarification

The older engine used a 5-second match trigger, while the optimized saved engine uses a 30-second trigger. This is not a "faster trigger" change. It reduces scheduling pressure and aligns the engine with the observed batch workload, especially because the saved log shows a batch already taking about 30 seconds during initial processing. A shorter trigger is only useful when the engine can reliably finish a batch within that window.

Evidence: Older match_engine.py, lines 347-364; optimized match_engine(1).py, start_matching_query near lines 3000+ and saved log lines 293-300.

11. Important Distinction: Performance vs Data Realism

The earlier investigation into high_priority_rows = 0 found a data-generation problem, not a Spark-performance problem. The current saved generator still chooses candidate and job independently in insert_application(). Because candidate/job skills are category-based and random pairing is not affinity-aware, most pairs can have weak skill overlap. That can suppress HIGH_PRIORITY matches even when the Spark pipeline itself is fast. Therefore, "zero high priority rows" must not be used as proof of poor pipeline performance.

Evidence: recruitment_lifecycle_generator.py, insert_application at lines 945-990; prior project diagnosis retained in the saved conversation/project context.

12. Final Conclusion

The project has undergone a substantial performance hardening pass. The dominant optimization theme is eliminating repeated work inside Spark: native JVM-side scoring, broadcast joins, column pruning, caching of reused intermediate DataFrames, fewer Spark actions, and Delta-native metrics. The second theme is runtime stability: explicit schemas, CDC deduplication, worker reuse, controlled local concurrency, pre-created Delta tables, and a Windows-safe output configuration. The generator has been simplified and made more efficient at the application layer, but per-row database commits remain the main unoptimized area there.

Most defensible performance claim: the saved optimized code is materially more efficient than the older match engine because it removes known high-cost execution patterns and explicitly prevents duplicate lineage evaluation. However, exact percentage improvement requires a controlled benchmark. The next benchmark should isolate (1) initial replay, (2) steady-state batches, and (3) generator-only throughput.

Evidence: Consolidated from the saved artifacts and runtime log; no external claims or unverifiable benchmark percentages were added.

13. Quantitative Performance Evidence

This section uses the actual batch metrics captured in the saved 25 Aug 2026 Spark log. The figures are measured observations, not simulated benchmarks.

13.1 Measured batch-time results

Batch	Incoming rows	Scored rows	Batch time	Rows/sec	30-sec trigger	Trigger status
0	46,913	33,675	85.163 s	550.86	30 s	+55.163 s over
1	17,518	5,037	38.999 s	449.19	30 s	+8.999 s over
2	15,395	2,891	29.382 s	523.96	30 s	0.618 s under
3	13,488	985	25.517 s	528.59	30 s	4.483 s under
1-3 avg	15,467	2,971	31.299 s	500.58	30 s	1.299 s over avg

Key observation: after the large initial replay, the measured batches reached approximately 25.5-29.4 seconds, which is around the configured 30-second trigger target. Batch 2 was 0.618 seconds under the trigger and Batch 3 was 4.483 seconds under it.

13.2 What changed numerically

Comparison	Before / reference	After / observed	Change	Interpretation
Batch wall-clock time	Batch 1: 38.999 s	Batch 3: 25.517 s	-13.482 s (-34.6%)	Later steady-state batch completed materially faster than Batch 1, while workload size also declined.
Throughput	449.19 rows/s	528.59 rows/s	+79.40 rows/s (+17.7%)	Per-batch throughput improved from Batch 1 to Batch 3.
Batch 0 vs Batch 3	85.163 s	25.517 s	-59.646 s (-70.0%)	Large raw reduction, but Batch 0 had 46,913 rows vs 13,488, so this is not a controlled 70% speedup claim.
30-second SLA	85.163 s	25.517 s	From +55.163 s to -4.483 s	The workload moved from far behind the trigger to slightly ahead by Batch 3.

Important benchmark caveat: these batches are not identical workloads. Input volume fell from 46,913 rows in Batch 0 to 13,488 rows in Batch 3. Therefore, the most defensible statement is that the optimized pipeline was observed running at ~25.5-29.4 s for later batches and ~501 rows/s on average across Batches 1-3; a formal percentage speedup requires identical before/after workloads.

13.3 Where the time is going

Batch	Pending write	Resolved check	Scoring write	High-priority check
0	15.525 s	8.236 s	12.205 s	1.224 s
1	19.550 s	1.446 s	2.520 s	2.581 s
2	7.478 s	3.060 s	2.743 s	0.246 s
3	11.229 s	2.054 s	1.309 s	0.110 s

The largest repeated cost visible in the saved metrics is pending-state write time, ranging from 7.478 to 19.550 seconds in Batches 1-3. This supports the report recommendation to optimize pending Delta persistence next.

13.4 Performance impact by optimization

Optimization	Numeric / observable evidence	Expected effect	Confidence
Cache resolved	Avoids recomputing the join/filter lineage for both emptiness check and scoring	Cuts repeated join work	High

Cache scored	Avoids recomputing full scoring before HIGH_PRIORITY check	Cuts repeated scoring work	High
Broadcast joins	Candidate dimension ~0.40-0.44 MB; job dimension ~0.91-1.04 MB in logged batches	Keeps small dimensions local to tasks and avoids a larger join strategy	High
Column pruning	Only 7 candidate columns + 10 job columns carried into enrichment	Reduces bytes read/cached/broadcast	High
Native Spark SQL scoring	Removes row-by-row Python UDF boundary	Reduces serialization/Python-worker overhead	Very high
Avoid extra unresolved counts	Unresolved row metrics taken from Delta commit metrics	Removes a full extra scan/action	High
Delta optimizeWrite + autoCompact	Enabled in Spark session	Can reduce small-file pressure on repeated writes	Medium; verify via files_written

13.5 Practical throughput target for this project

Current observed reference: ~500.6 input rows/second across Batches 1-3, with an average batch duration of 31.3 seconds. A practical next target is to consistently stay below the 30-second trigger for the expected steady-state batch size, then push toward 600-800 rows/second after generator-side batching and pending-write optimization. The 600-800 rows/second figure is a proposed engineering target, not a measured result.